CreditSense

AI-Assisted Credit Risk Assessment Platform

A Project Report

Submitted by

Mitul Bhatia	2401010279
Vaibhav	2401010486
Sarvjeet	2401010424
Hardik	2401010174

Contents

Abstract **2**

1 Executive Summary **2**

2 System Overview **3**

2.1 Technology Stack 3

2.2 Why the Architecture is Professional 3

3 System Architecture and Data Flow **4**

3.1 Architecture Diagram 4

3.2 Flow Interpretation by Layer 5

3.3 ONNX Migration Impact 5

4 Implementation Modules **5**

5 Contribution Matrix **6**

6 Git LFS and Artifact Management **6**

6.1 Why Git LFS Was Required 6

6.2 Project LFS Strategy 6

6.3 Operational Workflow Used 7

6.4 Result 7

7 Validation and Submission Readiness **7**

8 Conclusion and Forward Path **7**

Abstract

CreditSense addresses a core fintech challenge: producing fast credit decisions without sacrificing regulatory compliance or explainability. Traditional underwriting is slow and manual, while unconstrained LLM systems can hallucinate policy logic and create audit risk.

This project implements a modular multi-agent architecture using LangGraph, FastAPI, Streamlit, deterministic policy computation, and Chroma-based retrieval with ONNX embeddings. The system combines conversational intake, regulatory context retrieval, explicit numeric risk scoring, and bilingual report generation in one controlled pipeline.

The resulting platform is submission-ready, operationally clear, and structured for practical deployment on constrained infrastructure.

1 Executive Summary

Problem Statement

Financial institutions need explainable and reproducible loan-assessment workflows. Purely manual processes are slow, and purely generative systems can violate compliance expectations.

Project Objective

Design and implement a professional-grade, auditable underwriting assistant that provides deterministic decision support with context-grounded AI reporting.

Delivered Outcome

An end-to-end platform with user intake, regulatory retrieval, policy evaluation, bilingual reporting, PDF export, and production-oriented repository operations including Git LFS for large artifacts.

2 System Overview

2.1 Technology Stack

Frontend	Streamlit conversational interface with structured borrower intake and report rendering
Backend API	FastAPI service layer for ingestion, chat orchestration, and report endpoints
Orchestration	LangGraph stateful agent routing across guardrail, intake, retrieval, policy, and reporting nodes
Retrieval	Chroma persistent store with ONNXMiniLM_L6_V2 embeddings
Decision Core	Deterministic policy engine and heuristic risk scorer
Report Output	English and Hindi narratives with PDF export workflow

2.2 Why the Architecture is Professional

- Separation of concerns between interface, API orchestration, retrieval, scoring, and presentation.
- Deterministic numerical decision path for auditability.
- Controlled LLM usage for extraction and narrative generation, not final policy authority.
- Deployment-aware design for free-tier and low-resource environments.

3 System Architecture and Data Flow

3.1 Architecture Diagram

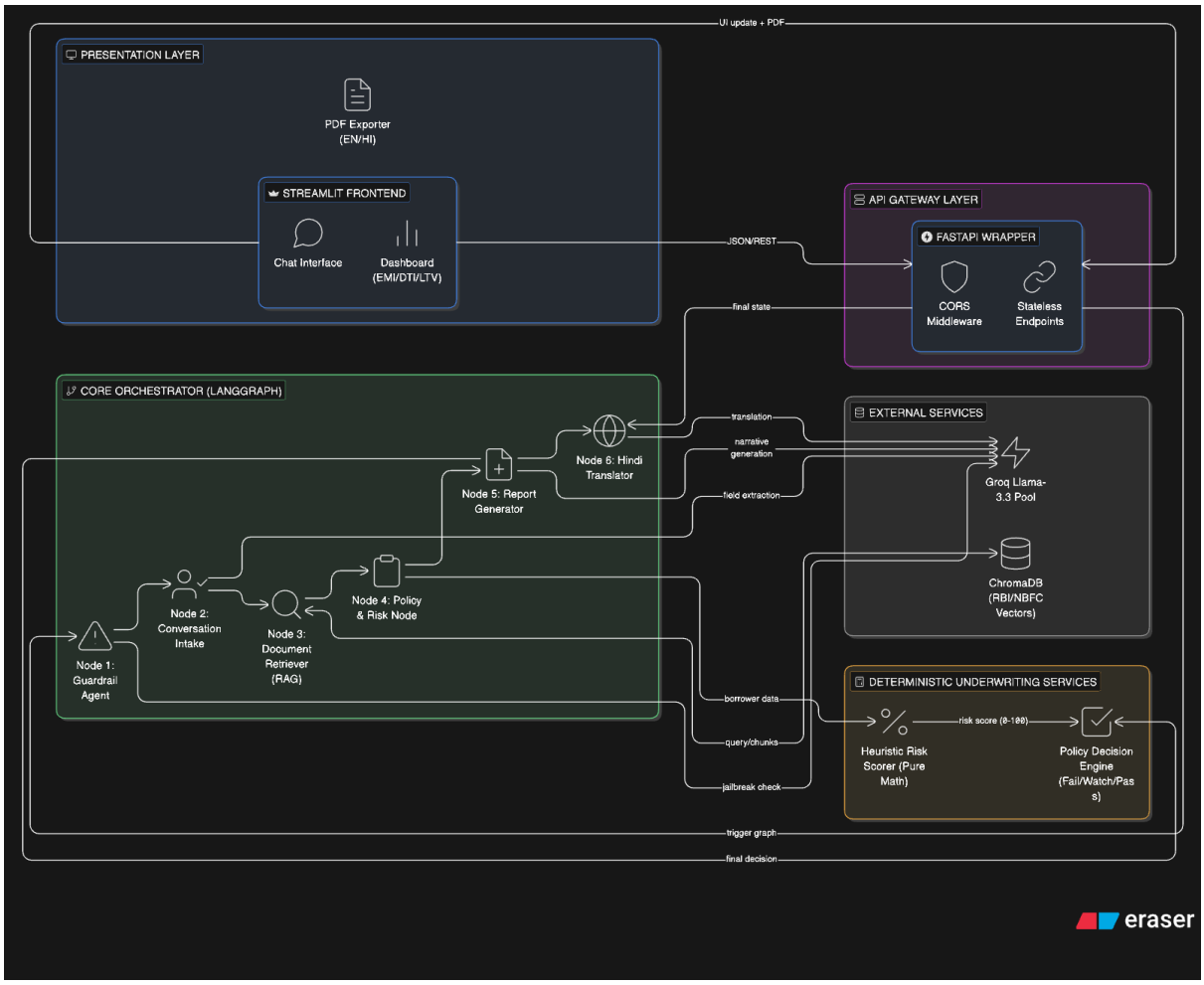

Figure 1: CreditSense tri-layer architecture used for final submission

3.2 Flow Interpretation by Layer

Presentation Layer	User interaction, profile capture, and report delivery through Streamlit and PDF exporter
API Gateway Layer	Stateless request handling, schema enforcement, and controlled route execution
Core Orchestrator	Multi-agent graph with explicit state transitions across intake, retrieval, policy, and translation
External Services	LLM pool and Chroma retrieval backend for regulatory grounding
Deterministic Services	Risk scoring and final decision logic using transparent mathematical rules

3.3 ONNX Migration Impact

Early versions relying on full sentence-transformers and PyTorch introduced high memory pressure in constrained environments. The migration to ONNXMiniLM_L6_V2 through Chroma maintained semantic retrieval quality while significantly reducing runtime footprint and startup overhead.

4 Implementation Modules

Module	Layer	Responsibility
Guardrail Agent	**Orchestrator**	Filters unsafe or irrelevant prompts before business processing
Conversation Intake	**Orchestrator**	Extracts borrower profile fields from natural language turns
Retriever Service	**Retrieval**	Queries Chroma collection and builds citation-ready context chunks
Policy Engine	**Deterministic Logic**	Computes DTI/LTV-driven outcomes and rules-based classification
Risk Scorer	**Deterministic Logic**	Produces transparent weighted risk score from borrower attributes
Report Generator	**Reporting**	Converts numeric outcomes into structured human-readable decision rationale
Translation Node	**Reporting**	Produces Hindi-compatible output for multilingual accessibility
PDF Exporter	**Delivery**	Generates downloadable final reports for submission and operations

5 Contribution Matrix

This matrix formalizes team ownership and deliverables for assessment and viva review.

Member	Primary Ownership	Key Deliverables
Mitul	**Project Lead and Full-Stack Development**	End-to-end architecture design, FastAPI backend, Streamlit integration, LangGraph orchestration, retrieval-service wiring, deployment flow, and final technical integration.
Vaibhav	**Technical Documentation and Report Strategy**	Structured technical documentation set, architecture write-ups, submission report curation, and narrative consistency across deliverables.
Sarvjeet	**Media and Presentation Lead**	Visual communication assets, final presentation deck, system-flow translation for audience-friendly explanation, and demo walkthrough packaging.
Hardik	**Product Ideation and Strategy**	Problem framing, requirements articulation, policy-threshold framing guidance, and solution direction aligned with practical user adoption.

6 Git LFS and Artifact Management

6.1 Why Git LFS Was Required

This project contains large binary artifacts, especially vector-store database files used by Chroma. Standard Git is inefficient for large mutable binaries and can fail or bloat repository history when these artifacts are versioned directly.

6.2 Project LFS Strategy

- Canonical milestone path normalized to `MILESTONE_2/`.
- Large vector-store artifacts tracked with Git LFS under `MILESTONE_2/creditsense/chroma_stor`
- Duplicate accidental path with spaces was removed to avoid dual database stores.

6.3 Operational Workflow Used

```
brew install git-lfs
git lfs install
git add .gitattributes
cd MILESTONE_2/creditsense/chroma_store
git add chroma.sqlite3
git commit -m "Track Chroma store with Git LFS"
git push origin main
```

6.4 Result

Repository operations remained stable while retaining required large project assets for reproducible demonstrations and submission packaging.

7 Validation and Submission Readiness

Validation Item	Outcome
Architecture Included	Embedded directly in this report from the official project diagram asset
Contribution Matrix	Included as a formal table with role ownership and deliverables
Modular Report Structure	Organized into executive summary, architecture, modules, matrix, LFS, and readiness sections
Professional Formatting	Consistent typography, color hierarchy, tabular summaries, and structured section flow
PDF Generation Ready	TeX source organized for reproducible PDF build and final submission export

8 Conclusion and Forward Path

CreditSense demonstrates a practical, auditable, and presentation-grade approach to AI-assisted underwriting. The system balances deterministic policy enforcement with contextual AI capabilities, while repository operations were hardened through Git LFS for large data assets.

Immediate future enhancements include:

- stronger authentication and role-based access controls,
- deeper monitoring and evaluation instrumentation,
- expanded benchmarking with larger historical credit datasets,
- automated CI checks for report and docs validation.